RNX2DB Optimization Summary

Changes Made to Codebase

All optimizations so far were implemented by Kathryn Butler. Any optimizations made by Joseph Schaab are not covered in this document at this time.

Station Position Auto-Detected (config.yaml)

Original:

```
station:
  id: 'ALIC'
  info: [-4052052.7072, 4212835.9827, -2545104.6244]
```

Optimized:

```
station:
  # if the station position can't be found in the obs header, manually
input it here
  info: [NULL, NULL, NULL]
```

rnx2db.py

```
pos_str = header.get('APPROX POSITION XYZ', None)
# if we can find APPROX POSITION XYZ and it's not 0, 0, 0
if pos_str is not None and not all(float(x) == 0 for x in
pos_str.split()):
    self.station_info = [float(x) for x in pos_str.split()]
```

The original code had to have someone manually input the position. Now the file looks for APPROX POSITION XYZ. If it doesn't exist or is 0,0,0, it will throw an error:

```
14:07:10 - ==================================
14:07:10 - GNSS RINEX Processing Started
14:07:10 - ==================================
14:07:10 - Loading config...
14:07:10 - Station position not found in header or config!
14:07:10 - Elevation and azimuth calculations will be inaccurate until station position is provided.
14:07:10 - Please update the position in the config file or make sure the Rinex header contains APPROX POSITION XYZ.
14:07:10 - Closing the program...
PS C:\Projects\optimizing-research-software-codes\gnss_python-main>
```

If it does not find it but the position is set in the config file, the program will output a warning to remind the user to make sure the config file's positions are set correctly:

```
14:09:12 - ==================================
14:09:12 - GNSS RINEX Processing Started
14:09:12 - ==================================
14:09:12 - Loading config...
14:09:12 - APPROX POSITION XYZ not found in observation header. Using station position from config file...
14:09:12 - Make sure the config file is up to date or the Rinex observation header has an accurate APPROX POSITION XYZ.
14:09:19 - Config loaded successfully! Processing RINEX files...
14:09:20 - Files processed! Calculating positions using 8 cores...
```

Relative Paths (config.yaml)

Original:

```
singlefile:
    date: '2024-07-10'
    observation:
      -
C:\Projects\gnss_python-main\Data\ALIC00AUS_R_20190910000_01D_30S_MO.19o
    navigation:
      -
C:\Projects\gnss_python-main\Data\BRDC00IGS_R_20190910000_01D_MN.rnx
  output_dir: C:\Projects\gnss_python-main\output\ALIC\compactdb
```

Optimized:

```
  singlefile:
    observation:
      - .\observation\NOME1190_rnx2.25O
    navigation:
      - .\navigation\NOME1190.25P
  output_dir: .\output
```

Now you don't have to include the absolute paths, station ID, or station coordinates unless the observation file doesn't have it. Also corrected a typo to turn *obs_size_threashold* into *obs_size_threshold*.

Leap Seconds (config.yaml)

Original:

```
etc:
  disable_parallel: false
  leap_second:
    - 1981-06-30
    - 1982-06-30
    - 1983-06-30
    - ...
```

Optimized:

```
etc:
  disable_parallel: false
```

Removed leap seconds from the config file and added *_leap_seconds* variable to rnx2db.py. That way, users don't need to worry about maintaining that list themselves and can't accidentally break it as easily.

MJS Timestamp Calculation

Original:

```
self.obs['MJS'] = self.obs.apply(lambda x: self.to_mjd(x.name[1]), axis=1)
```

Optimized:

```
  def add_mjd_column(self, df, apply_time_adjustments=True):
```

The original version calculated MJS timestamps row-by-row, which takes a long time. The optimized version uses numpy operations that work on all the rows at the same time, making it a lot faster.

More Parallel Chunks

Original:

```
      print(f'Using {num_procs} cores to calculate positions.\nIf you
encounter memory errors, reduce num_cores and try it again or disable
parallel')
      obs_partition = np.array_split(self.obs, num_procs)
```

Optimized:

```
      logging.info(f'{GREEN}Files processed!{RESET} Calculating positions
using {num_procs} cores...')
      chunks = num_procs * 4
      obs_partition = np.array_split(self.obs, chunks)
```

The new version splits up the observation data into finer chunks and uses *imap* instead of *map*, which allows the progress bar to update while the chunks are completing.

Elevation/Azimuth Vectorized

Original:

```
elevation, azimuth =
compute.calculate_elevation_azimuth(np.array(self.config['station']['info'
]), np.array([item.PosX, item.PosY, item.PosZ]))
```

Optimized:

```
el_az = compute.calculate_elevation_azimuth_batch(recv, sat_positions)
```

The original function called *compute.calculate_elevation_azimuth()* once per row, which is VERY costly in Python. The new version simply calls *compute.calculate_elevation_azimuth_batch()* once on the entire dataset and returns a numpy array. This is much more efficient because it computes all the elevations and azimuths in a single vectorized pass.

Improved Output Row Building

Original:

```
fp.write(f'{item.Index[prn_index]},{item.Index[dt_index]},{item.MJS},{item
.ClkCorr},{item.PosX},{item.PosY},{item.PosZ},{azimuth},{elevation},{r[0]}
,{r[1]},{r[2]},{r[3]},{r[4]},{r[5]}\n')
```

Optimized:

```
output_df = pd.DataFrame(rows, columns=_OUTPUT_COLUMNS)
output_df.to_csv(output_file, index=False, na_rep='nan')
```

The original would write to the file for every single input line. This is incredibly slow, and can cause a partial file if something crashes mid loop. The new one bundles up all the rows into an output dataframe and then writes everything to the csv in one go.

Better User Interface/Output While Running

Original:

```
print(f'Using {num_procs} cores to calculate positions.\nIf you encounter
memory errors, reduce num_cores and try it again or disable parallel')
```

Optimized:

```
logging.info(f'{GREEN}Files processed!{RESET} Calculating positions using
{num_procs} cores...')
tqdm(pool.imap(self.get_satellite_position_process_worker, obs_partition),
     total=len(obs_partition),
     desc="Calculating satellite positions",
     bar_format="{desc}: {percentage:.0f}% |{bar}| {n_fmt}/{total_fmt}
complete | Duration: {elapsed}s | ETA: {remaining}s ")
```

The original code used print statements that often got lost in all the warnings and errors that'd show up. The optimized version uses *logging.info* for a better understanding of timing, color-coded success and failure messages, and total duration at the end.

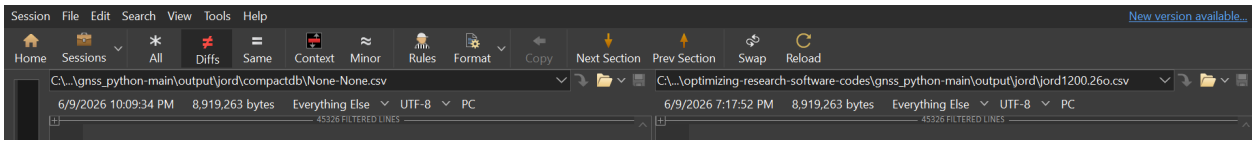
Performance and Correctness Results

Methodology

Both the original and optimized versions were run on the same datasets on the same machine, with limited major processes running. Runtimes were recorded from the logged outputs at the end of the runs. All outputs are confirmed to be identical, as shown in the following screenshots. One thing I did not have the opportunity to do was record more timing data for the original software, because the software had no timing functionality when we received it so it would have to be recorded by hand.

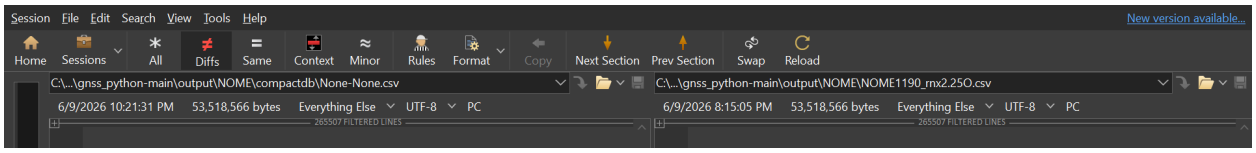
Jord1200

```
19:17:52 - Output saved to .\output\jord\jord1200.26o.csv.
19:17:52 - Processing completed!
19:17:52 - Total runtime: 27.44 seconds (0 minutes).
```



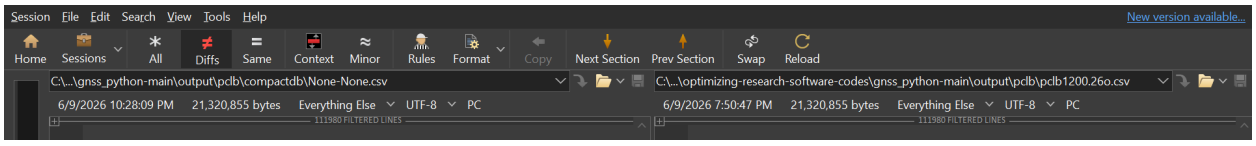
Nome 1190

```
20:15:05 - Output saved to .\output\NOME\NOME1190_rnx2.25O.csv.
20:15:05 - Processing completed!
20:15:05 - Total runtime: 96.70 seconds (2 minutes).
```



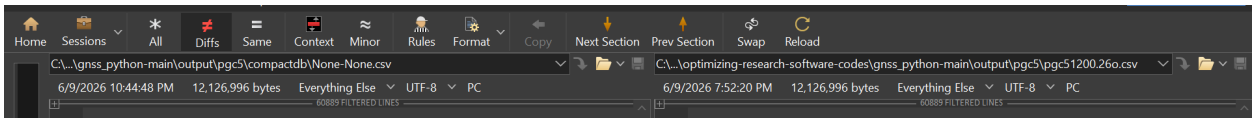
Pclb1200

```
19:50:47 - Output saved to .\output\pclb\pclb1200.26o.csv.
19:50:47 - Processing completed!
19:50:47 - Total runtime: 98.71 seconds (2 minutes).
```



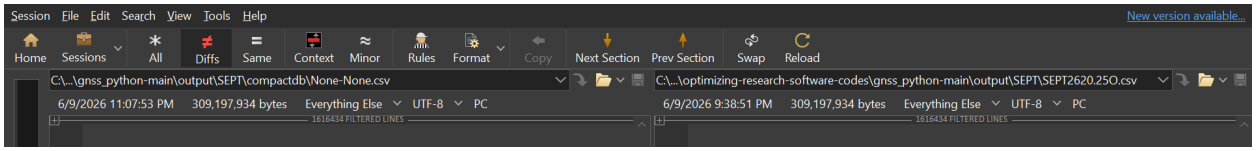
Pgc51200

```
19:52:20 - Output saved to .\output\pgc5\pgc51200.26o.csv.
19:52:21 - Processing completed!
19:52:21 - Total runtime: 32.88 seconds (1 minutes).
```



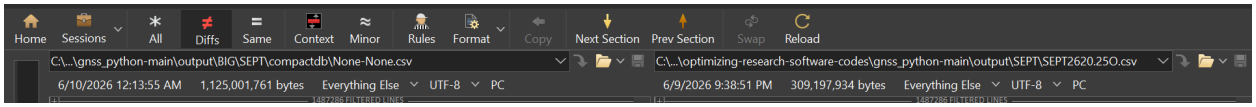
Sept2620

```
21:38:51 - Output saved to .\output\SEPT\SEPT2620.25O.csv.
21:38:51 - Processing completed!
21:38:51 - Total runtime: 571.83 seconds (10 minutes).
```



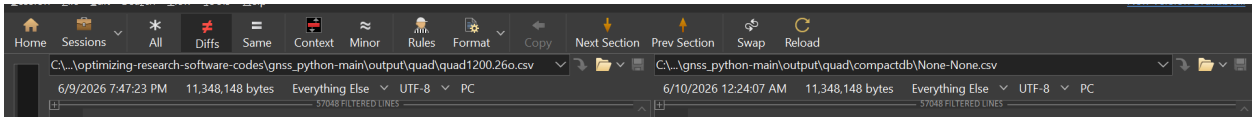
Sept2620-2627

```
21:14:27 - Output saved to .\output\SEPT\SEPT2620.25O.csv.
21:14:29 - Processing completed!
21:14:29 - Total runtime: 2570.67 seconds (43 minutes).
```



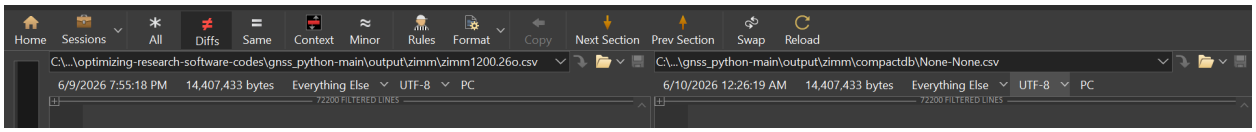
Quad1200

```
19:47:23 - Output saved to .\output\quad\quad1200.26o.csv.
19:47:23 - Processing completed!
19:47:23 - Total runtime: 31.70 seconds (1 minutes).
```



Zimm1200

```
19:55:18 - Output saved to .\output\zimm\zimm1200.26o.csv.
19:55:18 - Processing completed!
19:55:18 - Total runtime: 17.23 seconds (0 minutes).
```



How to Run

The same Anaconda system still works, but I used VSCode's venv environment because it made the whole process much easier for me. The GitHub website has more information about setting it up, as well as the official documentation.

Future Work & HPC

During the Engineering Expo, I met someone who asked me if we've considered using OSU's supercomputer before. I hadn't heard of it, so I reached out to a friend. She referred me to Rob Yelle, who is the cluster manager of the High Performance Computing (HPC) cluster. "The college provides CPU and GPU systems for general research and class use."

I think the HPC is an incredibly useful thing to look into, especially because this seems like the exact sort of project that could use the high performance of supercomputing. I asked my friend if she could share some tips on how to get started, which is attached in the HPC_how_to_use.pdf file. I think reaching out to this department would be a great next step to see if your calculations could be done much, much more quickly.